

Sintesi

Saverio Manco 1041566

framework utilizzati

Spring Boot si è rivelato molto adeguato al tipo di applicazione. Le annotazioni riducono al “minimo” il codice ripetitivo: le query piu' usate sono nate dai soli nomi dei metodi dei repository, senza scrivere SQL, e la dependency injection ha reso diciamo naturale separare controller, business logic e accesso ai dati. React ha retto bene la parte interattiva: il fatto che la vista si rigeneri da sola al cambiare dello stato ha evitato di dover aggiornare a mano il DOM, e la suddivisione in componenti mi ha permesso di riusare la stessa scheda per ogni appuntamento dell'elenco.

Le difficoltà che ho incontrato sono state soprattutto queste:

Versione di Spring Boot. Lo Spring Initializr propone ormai solo la versione 4, mentre il materiale del corso e gran parte della documentazione si riferiscono alla 3. Ho creato il progetto e poi riportato a mano la versione nel pom.xml alla 3.5.16, correggendo i nomi dei dependency starter, che nella 4 sono cambiati.

CORS con credenziali. Front-end e back-end girano su porte diverse, quindi ogni richiesta e' cross-origin. Far viaggiare il cookie di sessione ha richiesto credentials "include" lato client e allowCredentials lato server; con le credenziali abilitate il protocollo non consente piu' di autorizzare qualunque origine, che va quindi elencata esplicitamente

Filtro e risposte di errore. Il filtro di autenticazione blocca le richieste prima che raggiungano Spring MVC, dove vengono applicate le annotazioni @CrossOrigin: senza aggiungere a mano gli header CORS alla risposta 401, il browser impediva al codice JavaScript di leggerla e l'errore arrivava al front-end come un generico problema di rete.

Serializzazione delle entità. Restituire direttamente le @Entity portava a due problemi: la password finiva nel JSON e le relazioni bidirezionali generavano una ricorsione infinita. Ho introdotto i DTO, che risolvono entrambi e permettono di esporre anche dati calcolati, come i posti ancora disponibili.

Effetti asincroni in React. Cambiando filtro rapidamente, la risposta relativa alla materia precedente poteva arrivare dopo quella corretta e sovrascriverla. La funzione di cleanup degli effetti invalida le richieste ormai superate

Idee sul deployment

L'applicazione client-side verrebbe compilata in file statici e servita da un server web; l'applicazione server-side verrebbe impacchettata in un unico archivio eseguibile, dato che Spring Boot incorpora già il proprio server HTTP. La soluzione più semplice è metterle dietro allo stesso dominio tramite un reverse proxy, che inoltri le richieste con percorso /api al back-end e tutte le altre ai file statici: così le due parti condividono l'origine

Strumenti ulteriori

Sicurezza delle credenziali. Spring Security per gestire l'autenticazione e memorizzare le password come hash invece che in chiaro, come consentito dalla traccia in questa fase.

Invio di email. Un servizio SMTP per recapitare davvero il link della videochiamata e gli avvisi: oggi vengono mostrati nell'applicazione, ma non spediti realmente (e solo per idea di come potrebbe essere tipo)

Aggiornamenti in tempo reale. WebSocket, o in alternativa un'interrogazione periodica, perché oggi lo studente vede l'esito della propria richiesta solo ricaricando la pagina.

Test automatici. Test di unità sui service, dove vivono le regole di business, e test di integrazione sulle route.

Problematiche in un utilizzo

Traffico di rete. L'elenco degli appuntamenti include per ciascuno tutor, materia, argomenti e materiali: con poche decine di elementi non e' un problema (credo che larga scala servirebbero paginazione e una versione ridotta dei dati, riservando quelli completi al dettaglio)

Privacy. L'applicazione tratta dati personali: nome, email istituzionale, matricola e le motivazioni scritte dagli studenti, che possono rivelare difficolta' personali. Ho applicato il principio di minimizzazione dove possibile ma servirebbero anche una politica di conservazione dei dati e la possibilita' di cancellare il proprio profilo (credo che la privacy sia un punto fondamentale di questo progetto)

Sicurezza. Oltre alle password da cifrare, servirebbero una protezione piu severa e un limite ai tentativi di accesso

Accessibilita'

Sul piano tecnico ogni campo dei moduli e' associato alla propria etichetta, i messaggi di errore vengono annunciati dai lettori di schermo, le finestre modali sono dichiarate come tali e collegate al proprio titolo, si chiudono con il tasto Esc e l'intera applicazione e' percorribile da tastiera con un contorno di messa a fuoco ben visibile

Sul piano dell'inclusivita', l'obiettivo dell'applicazione e' esso stesso inclusivo: rende accessibile a chiunque un aiuto che oggi circola solo attraverso reti di amicizie, penalizzando le matricole e gli studenti fuori sede. Restano alcuni limiti di cui essere consapevoli: l'interfaccia e' solo in italiano (nel eventuale pensavo ad uno switch in altre lingue), il sistema di valutazioni, se non gestito con attenzione, rischia di penalizzare chi inizia a fare tutoraggio e non ha ancora ricevuto giudizi